Sequence Labeling - Graded - RNN

November 6, 2026

1 Sequence Labeling with RNNs

This lab introduces Recurrent Neural Networks (RNNs) and their application to sequence labeling tasks, specifically Part-of-Speech (POS) tagging using the Brown corpus.

Key Concepts:

1. Recurrent Neural Networks (RNNs):
   - Neural networks designed to work with sequential data
   - Process input sequences one element at a time while maintaining a hidden state
   - Can handle variable-length sequences
   - Well-suited for tasks like language processing, time series analysis, and sequence labeling
2. Long Short-Term Memory (LSTM):
   - A sophisticated type of RNN that addresses the vanishing gradient problem
   - Contains specialized gates (input, forget, output) to control information flow
   - Better at capturing long-term dependencies in sequences
   - More stable training compared to vanilla RNNs
3. Sequence Labeling:
   - Task of assigning a categorical label to each element in a sequence
   - Examples include POS tagging, named entity recognition, and chunking
   - Each input token maps to exactly one output label
4. Part-of-Speech (POS) Tagging:
   - Linguistic task of marking words with their grammatical categories
   - Examples: noun, verb, adjective, determiner, etc.
   - Context-dependent: same word can have different POS tags based on usage
   - Essential for many downstream NLP tasks
5. Key Components in Implementation:
   - Word Embeddings: Dense vector representations of words
   - LSTM Layer: Processes the sequence and captures contextual information
   - Output Layer: Maps LSTM outputs to tag probabilities
   - Cross-Entropy Loss: Standard loss function for classification tasks

In this lab, we'll implement a complete sequence labeling system using PyTorch, demonstrating how to: - Process and prepare sequential data - Build an RNN-based model architecture - Train the model using backpropagation through time - Evaluate the model's performance on a held-out test set

The Brown corpus serves as our dataset, providing pre-tagged sentences that we'll use for training and testing our model.

1.1 Install and Import Required Libraries

In some environments (such as Google Colab), you may need to install nltk first. If you are running locally and have nltk already, you can omit the installation.

[1]:
```
!pip install nltk
```

```
WARNING: The directory '/home/jovyan/.cache/pip' or its parent directory is
not owned or is not writable by the current user. The cache has been disabled.
Check the permissions and owner of that directory. If executing pip with sudo,
you should use sudo's -H flag.
Requirement already satisfied: nltk in /opt/conda/lib/python3.10/site-
packages (3.9.1)
Requirement already satisfied: click in /opt/conda/lib/python3.10/site-packages
(from nltk) (8.1.3)
Requirement already satisfied: joblib in /opt/conda/lib/python3.10/site-packages
(from nltk) (1.2.0)
Requirement already satisfied: regex>=2021.8.3 in
/opt/conda/lib/python3.10/site-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in /opt/conda/lib/python3.10/site-packages
(from nltk) (4.67.1)
WARNING: Running pip as the 'root' user can result in broken permissions
and conflicting behaviour with the system package manager, possibly rendering
your system unusable. It is recommended to use a virtual environment instead:
https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know
what you are doing and want to suppress this warning.

[notice] A new release of pip is
available: 25.0.1 -> 26.3
[notice] To update, run:
pip install --upgrade pip
```

[2]:
```
"""
RNN-based Sequence Labeling (POS Tagging with Brown Corpus)
"""

import torch
from torch import nn
from torch import optim
import random

import nltk
nltk.download('brown')
```

```
nltk.download('universal_tagset')
from nltk.corpus import brown

# Utility function to count the number of trainable parameters in a model
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

# Utility function to print all model parameters and their gradients
def print_parameters(model):
    for name, param in model.named_parameters():
        print(name)
        print(param.data)
        print(param.grad)
```

```
[nltk_data] Downloading package brown to /home/jovyan/nltk_data…
[nltk_data]   Unzipping corpora/brown.zip.
[nltk_data] Downloading package universal_tagset to
[nltk_data]     /home/jovyan/nltk_data…
[nltk_data]   Unzipping taggers/universal_tagset.zip.
```

1.2 Data Preparation

Here, we are going to build a toy dataset from the Brown corpus for a simplified part-of-speech tagging task.

- Load the Brown corpus: We'll retrieve sentences, each of which is a list of (word, tag) pairs.
- Filter/Map Tags: Brown offers quite a few tags, but to keep it simpler, we'll use the universal tagset which has 12 coarse-grained POS categories.
- Split into Train and Test: We'll create a small train/test split so that we can train on one portion and test on another.
- Convert to Indices: We need to build a vocabulary for words and a label set for tags. We'll map each unique word to an integer index and each unique tag to a label index.

[3]:
```
################################################
# 2.1. Load and Inspect Brown Corpus Sentences
################################################
nltk_sentences = brown.tagged_sents(tagset='universal')  # Each sentence is␣
 ↪list of (word, tag) pairs
print("Total sentences in Brown (Universal Tagset):", len(nltk_sentences))
print("Example:", nltk_sentences[0])

########################################
# 2.2. Create a Small Subset of Data
########################################
# For demonstration, we'll use fewer sentences.
# You can expand this if you want more training data.
n_samples = 200  # Feel free to increase
sentences = nltk_sentences[:n_samples]
```

```
##########################################
# 2.3. Build Word and Tag Vocabulary
##########################################
word_to_idx = {}
tag_to_idx = {}
word_counter = 0
tag_counter = 0

for sent in sentences:
    for (word, tag) in sent:
        w = word.lower()  # convert to lowercase to reduce vocab size
        if w not in word_to_idx:
            word_to_idx[w] = word_counter
            word_counter += 1
        if tag not in tag_to_idx:
            tag_to_idx[tag] = tag_counter
            tag_counter += 1

print("Size of word vocabulary:", len(word_to_idx))
print("Size of tag set:", len(tag_to_idx))

##########################################
# 2.4. Convert Sentences into Tensor Form
##########################################
dataset = []
for sent in sentences:
    word_indices = []
    tag_indices = []
    for (word, tag) in sent:
        word_indices.append(word_to_idx[word.lower()])
        tag_indices.append(tag_to_idx[tag])
    dataset.append( (torch.tensor(word_indices, dtype=torch.long),
                     torch.tensor(tag_indices, dtype=torch.long)) )

######################################
# 2.5. Split into Train and Test
######################################
train_ratio = 0.8
train_size = int(len(dataset)*train_ratio)
train_data = dataset[:train_size]
test_data = dataset[train_size:]

print("Training samples:", len(train_data))
print("Test samples:", len(test_data))
```

```
Total sentences in Brown (Universal Tagset): 57340
```

```
Example: [('The', 'DET'), ('Fulton', 'NOUN'), ('County', 'NOUN'), ('Grand',
'ADJ'), ('Jury', 'NOUN'), ('said', 'VERB'), ('Friday', 'NOUN'), ('an', 'DET'),
('investigation', 'NOUN'), ('of', 'ADP'), ("Atlanta's", 'NOUN'), ('recent',
'ADJ'), ('primary', 'NOUN'), ('election', 'NOUN'), ('produced', 'VERB'), ('``',
'.'), ('no', 'DET'), ('evidence', 'NOUN'), ("''", '.'), ('that', 'ADP'), ('any',
'DET'), ('irregularities', 'NOUN'), ('took', 'VERB'), ('place', 'NOUN'), ('.',
'.')]
Size of word vocabulary: 1436
Size of tag set: 11
Training samples: 160
Test samples: 40
```

1.3 Define the RNN Model for Sequence Labeling

Our model will have:

- An Embedding layer: Converts word indices to dense embeddings.
- An LSTM: A unidirectional LSTM to process the sequence of embeddings.
- An Output Linear layer: Projects the hidden states to the number of possible tags.
- A Non-linear Activation (e.g., ReLU) if needed (often used inside or after LSTM outputs).

[5]:
```
#?nn.Embedding
```

[30]:
```
class SeqLabRNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim):
        super().__init__()

        # 3.1. Embedding Layer
        self.embedding = nn.Embedding(
            # TODO: fill in with vocab_size, embedding_dim
            # your code here
            #raise NotImplementedError
            vocab_size, embedding_dim
        )

        # 3.2. LSTM Layer
        # https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html
        # (input_size=embedding_dim, hidden_size=hidden_dim, num_layers=1,␣
 ↪bias=False, batch_first=False, etc.)
        self.lstm = nn.LSTM(
            # TODO: embedding_dim,
            # TODO: hidden_dim,
            # your code here
            embedding_dim,
            hidden_dim,
            #raise NotImplementedError
            num_layers=1,
            bias=False
```

```
        )

        # 3.3. Output Projection Layer
        self.output_layer = nn.Linear(
            # TODO: fill in with hidden_dim, output_dim
            # your code here
            hidden_dim, output_dim
            #raise NotImplementedError
        )

        # 3.4. (Optionally) a Non-linear activation
        self.relu = nn.ReLU()

    def forward(self, input_seq):
        # input_seq shape: [seq_len] (1D containing indices of words)

        # a) Embedding
        embeddings = self.embedding(input_seq)
        # embeddings shape: [seq_len, embedding_dim]
        #print(embeddings.shape, embeddings.squeeze().shape)

        # b) LSTM
        # TODO: call LSTM on embeddings (note LSTM expects [seq_len,␣
↪batch_size, embed_dim])
        # lstm_output, (h_n, c_n) =

        # your code here
        #raise NotImplementedError
        lstm_output, (h_n, c_n) = self.lstm(embeddings) #.squeeze())
        #print(lstm_output.squeeze().shape)

        # c) (Optionally) apply ReLU to LSTM output
        #    But typically we apply the Linear layer directly on the LSTM␣
↪outputs.

        # d) Output layer: transform LSTM outputs to tag predictions
        output = self.output_layer(lstm_output) #.squeeze())
        # output shape: [seq_len, output_dim]

        return output
```

[31]:
```
def test_seqlabrnn_init():
    # Initialize model with test parameters
    vocab_size = 1000
    embedding_dim = 64
    hidden_dim = 128
    output_dim = 10
```

```
    model = SeqLabRNN(vocab_size, embedding_dim, hidden_dim, output_dim)

    # Test embedding layer dimensions
    assert model.embedding.num_embeddings == vocab_size, "Embedding vocabulary␣
↪size incorrect"
    assert model.embedding.embedding_dim == embedding_dim, "Embedding dimension␣
↪incorrect"

    # Test LSTM layer dimensions
    assert model.lstm.input_size == embedding_dim, "LSTM input size incorrect"
    assert model.lstm.hidden_size == hidden_dim, "LSTM hidden size incorrect"
    assert model.lstm.num_layers == 1, "LSTM number of layers incorrect"
    assert model.lstm.bias == False, "LSTM bias setting incorrect"

    # Test output layer dimensions
    assert model.output_layer.in_features == hidden_dim, "Output layer input␣
↪size incorrect"
    assert model.output_layer.out_features == output_dim, "Output layer output␣
↪size incorrect"

    # Test forward pass with sample input
    seq_len = 5
    input_seq = torch.randint(0, vocab_size, (seq_len,))
    output = model(input_seq)
    assert output.shape == (seq_len, output_dim), f"Expected output shape␣
↪{(seq_len, output_dim)}, got {output.shape}"

    print("All tests passed!")

test_seqlabrnn_init()
```

```
All tests passed!
```

[32]:
```
def test_seqlabrnn_forward():
    """Test the forward method of SeqLabRNN"""
    vocab_size, embedding_dim, hidden_dim, output_dim = 100, 50, 64, 10
    model = SeqLabRNN(vocab_size, embedding_dim, hidden_dim, output_dim)

    # Create a sample input sequence
    seq_len = 8
    input_seq = torch.randint(0, vocab_size, (seq_len,))

    # Run forward pass
    output = model(input_seq)

    # Check output shape - should be [seq_len, output_dim]
```

```
    assert output.shape == (seq_len, output_dim)
    print("Forward pass test passed!")

test_seqlabrnn_forward()
```

```
Forward pass test passed!
```

1.4 Initialize the Model

Below we define:

vocab_size = total number of unique words. embedding_dim = dimension of word vectors. hidden_dim = dimension of LSTM hidden layer. output_dim = number of tags (unique labels).

[33]:
```
vocab_size = len(word_to_idx)
embedding_dim = 50   # can experiment
hidden_dim = 64      # can experiment
output_dim = len(tag_to_idx)

model = SeqLabRNN(vocab_size, embedding_dim, hidden_dim, output_dim)
print("Number of trainable parameters:", count_parameters(model))
```

```
Number of trainable parameters: 101699
```

1.5 Training Setup

1.5.1 Define Loss Function

For multi-class sequence labeling, a common choice is the cross-entropy loss (nn.CrossEntropyLoss). Note that nn.CrossEntropyLoss in PyTorch expects raw logits of shape [N, C] (not softmax-ed) where:

N is the total number of tokens (flattened across sequences), C is the number of classes. ### Define Optimizer We can use SGD or Adam. Here, we'll just use simple SGD.

[34]:
```
# Define loss and optimizer

# criterion = # TODO
# optimizer = # TODO: add the optimizer with model.parameters(), experiment␣
 ↪with different learning rates

# your code here
#raise NotImplementedError
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)
```

[35]:
```
def test_loss_optim():
    # Test the loss function and optimizer
    assert isinstance(criterion, nn.CrossEntropyLoss), "Loss function should be␣
 ↪CrossEntropyLoss"
```

```
    assert isinstance(optimizer, optim.SGD), "Optimizer should be SGD"
    assert optimizer.defaults['lr'] == 0.1, "Learning rate should be 0.1"

    # Check optimizer is connected to model parameters
    model_params = list(model.parameters())
    optimizer_params = list(optimizer.param_groups[0]['params'])
    assert len(model_params) == len(optimizer_params), "Optimizer should␣
 ↪contain all model parameters"

    for mp, op in zip(model_params, optimizer_params):
        assert mp is op, "Optimizer parameter references should match model␣
 ↪parameters"

    print("Loss function and optimizer test passed!")

test_loss_optim()
```

```
Loss function and optimizer test passed!
```

1.6 Training Loop

We will train for a fixed number of epochs. During each epoch:

- Set model to training mode: model.train()
- Zero the gradients: optimizer.zero_grad()
- Forward pass: Get the raw logits from the model for each token in the sentence.
- Reshape your outputs/labels if needed so that they match [N, C] and [N].
- Compute loss: Typically CrossEntropyLoss between your predictions and labels.
- Backprop: loss.backward()
- Update parameters: optimizer.step() In a real project, you might want to create mini-batches of sentences. Here, we'll do a simple loop over each sentence.

[40]:
```
epochs = 5  # feel free to increase

for epoch in range(epochs):
    print(f"### Epoch {epoch+1} ###")
    total_loss = 0.0
    model.train()

    for (x, y) in train_data:
        # a) Zero out gradients from previous step
        # TODO: optimizer.zero_grad()

        # your code here
        #raise NotImplementedError
        optimizer.zero_grad()
```

```
        # b) Forward pass: get raw scores for each token
        # TODO: model(x)

        # y_raw =

        # your code here
        #raise NotImplementedError
        y_raw = model(x)

        # y_raw shape = [seq_len, output_dim]
        # y shape = [seq_len]

        # c) (Optionally) flatten so CrossEntropyLoss can handle shape = [N, C]
        # Also ensure y is shape [N]
        # For example:
        # y_raw = y_raw.view(-1, output_dim)
        # y = y.view(-1)

        # d) Compute loss
        # TODO: criterion(...)
        # loss =

        # your code here
        #raise NotImplementedError
        y_raw = y_raw.view(-1, output_dim)
        y = y.view(-1)
        #print(y_raw.shape, y.shape)
        loss = criterion(y_raw, y)

        # e) Backprop
        # TODO: loss.backward()

        # your code here
        #raise NotImplementedError
        loss.backward()

        # f) Update parameters
        # TODO: optimizer.step()

        # your code here
        #raise NotImplementedError
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(train_data)
    print("Average train loss:", avg_loss)
```

```
### Epoch 1 ###
Average train loss: 0.5923228429164737
### Epoch 2 ###
Average train loss: 0.5482854454312474
### Epoch 3 ###
Average train loss: 0.5078992455150001
### Epoch 4 ###
Average train loss: 0.4707613807404414
### Epoch 5 ###
Average train loss: 0.4365397284098435
```

[41]:
```python
def test_training_loop():
    """Test a simple training loop implementation"""
    vocab_size, embedding_dim, hidden_dim, output_dim = 100, 50, 64, 10
    model = SeqLabRNN(vocab_size, embedding_dim, hidden_dim, output_dim)

    # Setup optimizer and loss
    optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
    criterion = nn.CrossEntropyLoss()

    # Create a mini training set
    mini_train_data = []
    for _ in range(3):
        seq_len = 5
        x = torch.randint(0, vocab_size, (seq_len,))
        y = torch.randint(0, output_dim, (seq_len,))
        mini_train_data.append((x, y))

    # Run a single training step on each sample
    initial_loss = 0
    model.train()
    for x, y in mini_train_data:
        optimizer.zero_grad()
        y_raw = model(x)
        loss = criterion(y_raw.view(-1, output_dim), y.view(-1))
        loss.backward()
        optimizer.step()
        initial_loss += loss.item()

    # Run another epoch to see if loss decreases
    final_loss = 0
    for x, y in mini_train_data:
        optimizer.zero_grad()
        y_raw = model(x)
        loss = criterion(y_raw.view(-1, output_dim), y.view(-1))
        loss.backward()
        optimizer.step()
```

```
        final_loss += loss.item()

    print(f"Initial average loss: {initial_loss/len(mini_train_data)}")
    print(f"Final average loss: {final_loss/len(mini_train_data)}")
    print("Training loop test passed!")

test_training_loop()
```

```
Initial average loss: 2.274908463160197
Final average loss: 2.27002747853597
Training loop test passed!
```

1.7 Testing/Evaluation

Typically, you would:

- Set model to eval mode (model.eval()).
- Turn off gradient computations with torch.no_grad().
- Loop over your test_data and accumulate predictions.
- Compute accuracy or other metrics (e.g. F1 for certain tagging tasks). Below is a simple snippet to see how one might evaluate token-level accuracy on the test set.

[42]:
```
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for (x, y) in test_data:
        logits = model(x)
        # shape: [seq_len, output_dim]

        # We pick the highest scoring tag
        predicted_tags = logits.argmax(dim=-1)

        total += len(y)
        correct += (predicted_tags == y).sum().item()

print(f"Test Accuracy: {correct/total:.4f}")
```

```
Test Accuracy: 0.7927
```

[ ]:
```

```